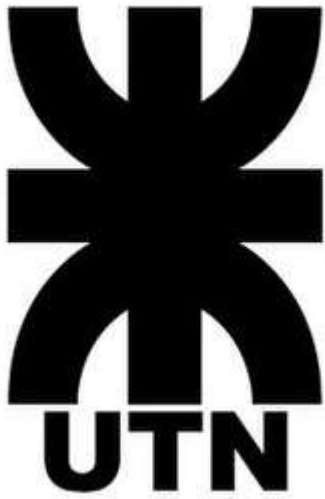




## **Universidad Tecnológica Nacional - Facultad Regional San Nicolás**

### **PROGRACION III – TRABAJO INTEGRADOR**

**Profesor coordinador:** Enferrel Ariel.

**Alumno:** Alex Austin Nahuel.

**Comisión N°:** 06

**Profesor tutor:** Giuliano Espejo Mezzabotta

**Link del video:** [https://youtu.be/A5Y7XRBFGy8?si=eUa\\_g6GJbj-8YUTn](https://youtu.be/A5Y7XRBFGy8?si=eUa_g6GJbj-8YUTn)

**Link de GitHub:**

<https://github.com/AlexNahuelAustin/Programacion-III-Integrador>

## **INTRODUCCION**

FoodStore es el sistema desarrollado como trabajo practico integrador de la materia programación III de la Universidad Tecnológica Nacional

El proyecto consiste en el desarrollo de una aplicación web e-commerce para la gestión de un catálogo de productos alimenticios, permitiendo a los usuarios realizar compras y a los administrar la gestión del inventario.

El objetivo principal del proyecto es aplicar conceptos fundamentales de desarrollo full-stack, integrando un Frontend interactivo y responsivo con un backend que maneje adecuadamente la lógica de negocio, validaciones y persistencia de datos. Esto incluye gestión de roles (ADMIN/USUARIO), autenticación, carrito de compras, historial de pedidos y reportes.

El sistema está diseñado para ser escalable, mantenible y demostrar el dominio de tecnologías como TypeScript, Vite, Java, JPA/Hibernate y H2 Database.

## Objetivos

### Objetivo General

El objetivo general es el desarrollo de una aplicación e-commerce funcional que va integrar el desarrollo del Frontend y backend

### Objetivos Específicos

- La implementación de un catálogo dinámico.
- Desarrollar un sistema de autenticación y protección de rutas.
- Crear el carrito de compra y con persistencia de LocalStorage.
- Diseñar el panel de administrador.
- Implementar el CRUD en el backend.
- Garantizar la persistencia de datos con JPA/Hibernate y H2 Database

## Diseño Arquitectónico

FoodStore implementa su frontend (TypeScript + vite) con módulos independientes organizado en pages, types, utils y assets; y backend (Java + JPA/Hibernate) con capas de model, repository, menus y util.

### Frontend

```
src/  
├── pages/ # Páginas: auth, store, client, admin  
├── types/ # Interfaces TypeScript  
├── utils/ # Servicios: auth.ts, dataService.ts, navigate.ts  
├── assets/ # Imágenes y recursos  
├── style.css # Estilos globales  
└── main.ts # Routing y protección de rutas
```

### Backend

```
src/main/java/com/tp/jpa/  
├── exceptions/ # Excepciones personalizadas  
├── init/ # DataLoader  
├── menus/ # Lógica de negocio  
├── model/ # Entidades y DTOs  
├── repository/ # Acceso a datos  
└── util/ # Configuración JPA
```

## **Tecnologías Utilizadas**

### **Frontend**

- TypeScript: Tipado estático para mayor seguridad
- Vite: Build tool moderno
- pnpm: Gestor de paquetes
- HTML5 y CSS3: Maquetación y estilos responsivos

### **Backend**

- Java 21: Lenguaje de programación
- JPA/Hibernate 6.5.0: ORM para mapeo objeto-relacional
- H2 Database 2.2.224: Base de datos
- Lombok 1.18.38: Generador de código boilerplate
- Gradle: Gestor de dependencias y build

## **Funcionalidades Principales**

El sistema cuenta con funcionalidades tanto en el frontend como en el backend. En el frontend, los usuarios pueden autenticarse mediante login y registro, explorar un catálogo dinámico con filtros por categorías y buscador, gestionar un carrito de compras con la posibilidad de modificar cantidades, realizar checkout con datos de entrega, y acceder al historial de pedidos. El panel administrativo proporciona un dashboard con estadísticas y tablas de los productos, categorías y de los pedidos, con protección de rutas según rol.

En el backend, se implementa CRUD completo de usuarios, productos, categorías y pedidos con persistencia de datos en H2 Database.

### **Frontend**

- Autenticación: Login y registro con validación de credenciales
- Catálogo dinámico: Visualización de productos con filtro por categorías y buscador
- Carrito de compras: Agregar/quitar productos, modificar cantidades, calcular totales
- Checkout: Modal de confirmación con datos de entrega y método de pago
- Historial de pedidos: Visualización de pedidos realizados con detalles
- Panel administrativo: Dashboard con estadísticas y tablas de categorías, productos y los pedidos
- Protección de rutas: Acceso restringido según rol.

## **Backend**

- Gestión de usuarios: Manejo de CRUD.
- Gestión de productos y categorías: CRUD completo.
- Sistema de pedidos: Manejo de CRUD y la creación atómica de un pedido.
- Persistencia de datos: Almacenamiento en H2 Database.
- Reportes JPQL: Consultas de productos, pedidos y totales facturados.

## **Decisiones de Diseño**

El desarrollo del diseño fue siguiendo la separación de responsabilidades para que su mantenibilidad y escalabilidad sea más fácil de arreglar. En cual va a ser una aplicación desacoplada donde frontend y backend funcionan de manera independiente, permitiendo futuras integraciones con APIs reales.

Se implementó un patrón de arquitectura de capas que facilita el testing y el mantenimiento del código. En el frontend se utilizó localStorage para persistencia de datos, evitando complejidades iniciales. En el backend se aplicó repositorios para centralizar el acceso a datos, y se crearon excepciones personalizadas para capturar errores específicos del negocio.

Se implementó CRUD completo para usuarios, productos, categorías y pedidos, validando stock y aplicando cascadas de eliminación donde corresponde.

## **Problemas Encontrados y Soluciones**

Durante el desarrollo tuve varios problemas. Uno de ellos fue en la función checkAuthUser() en los archivos del panel administrativo (adminHome, categories, products, orders) que tenía mala redirección de rutas y faltaban en algunas partes, por lo que el usuario podía entrar a algunas secciones del panel de administrador sin autorización. Esto fue solucionado corrigiendo las rutas y agregando la protección de rutas adecuada.

Otro problema fue en el backend, donde la función altaPedido() presentaba un problema donde solo se guardaba el primer producto de la lista en los detalles del pedido, ignorando los demás items del carrito. Esto se solucionó revisando la lógica de iteración y asegurando que todos los productos se agregaran correctamente.

También se presentó un inconveniente al listar los pedidos donde no se mostraban los datos. La solución fue modificar el PedidoDTO, quitando el atributo que traía la cantidad del detalle, solucionando así el problema de visualización.

## **Conclusiones**

En el trayecto del desarrollo del trabajo integrador me permitió aplicar y consolidar conceptos esenciales para el desarrollo full-stack. Creando una aplicación frontend y backend, donde se demuestra la importancia de la separación de responsabilidades y la arquitectura escalable.

En cual durante el camino se presentaron desafíos como incompatibilidades de librerías, errores de lógica en funciones críticas y problemas de integración entre capas. Donde cada problema fue resuelto y fortaleció la comprensión de las tecnologías utilizadas y aplicar buenas practicas.

Constituyendo una base para en un futuro agregar mejoras como APIs reales, autenticación más robusta, sistema de pagos y despliegue en producción.

**Fuentes:**

- Documentación oficial de TypeScript: <https://www.typescriptlang.org/>
- Documentación oficial de Vite: <https://vitejs.dev/>
- Documentación oficial de Java: <https://docs.oracle.com/en/java/>
- Documentación oficial de Hibernate: <https://hibernate.org/>
- Documentación oficial de Lombok: <https://projectlombok.org/>
- <https://developer.mozilla.org/es/>
- Apuntes y guías de la cátedra Programación III (UTN)